

CO3-AT1-Debugging

← Back

QUESTIONS

Q1
Producer-Consumer with
BlockingQueue
25 marks**Q2**
Finally Block Overriding Return
Values
25 marks**Q3**
try-with-resources Misuse
25 marks**Q4**
wait() and notify() Coordination
25 marks

4/4 answered

Q1 Producer-Consumer with BlockingQueue

25 marks

```
import java.util.concurrent.*;

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        Thread producer = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                queue.add(i);
            }
        };

        Thread consumer = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                int val = queue.poll();
                System.out.println("Consumed: " + val);
            }
        };

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("Queue size: " + queue.size());
    }
}
```

Your Code

```
1 import java.util.concurrent.*;
2 public class ProducerConsumerDemo {
3     public static void main(String[] args) throws InterruptedException
4     {
5         BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
6         Thread producer = new Thread() -> {
7             for (int i = 0; i < 10; i++) {
8                 try {
9                     queue.put(i);
10                } catch (InterruptedException e) {
11                    e.printStackTrace();
12                }
13            }
14        };
15        Thread consumer = new Thread() -> {
16            for (int i = 0; i < 10; i++) {
17                try {
18                    int val = queue.take();
19                    System.out.println("Consumed: " + val);
20                } catch (InterruptedException e) {
21                    e.printStackTrace();
22                }
23            }
24        };
25        producer.start();
26        consumer.start();
27    }
28 }
```

Input

stdin input

Output

```
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
```

QUESTIONS

Q1
Producer-Consumer with
BlockingQueue
25 marks

Q2
Finally Block Overriding Return
Values
25 marks

Q3
try-with-resources Misuse
25 marks

Q4
wait() and notify() Coordination
25 marks

4/4 answered

Q2 Finally Block Overriding Return Values

25 marks

```
public class FinallyReturnDemo {
    public static int riskyDivision(int a, int b) {
        try {
            int result = a / b;
            return result;
        } catch (ArithmeticException e) {
            return -1;
        } finally {
            return 0;
        }
    }

    public static void main(String[] args) {
        try {
            System.out.println(riskyDivision(10, 0));
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        int x = 5;
        try {
            x = 10;
            throw new RuntimeException("fail");
        } finally {
            x = 20;
        }
        System.out.println(x); error (unreachable code)

        int y;
        try {
            y = compute();
        } finally {
        }
        System.out.println(y); might not be initialized
    }

    static int compute() {
        return 42;
    }
}
```

Your Code

```
1 public class FinallyReturnDemo {
2     public static int riskyDivision(int a, int b) {
3         try {
4             int result = a / b;
5             return result;
6         } catch (ArithmeticException e) {
7             return -1;
8         } finally {
9             return 0;
10        }
11    }
```

Input

stdin input

Output

```
Result of riskyDivision(10, 0): 0
Caught exception: fail
Value of x after finally: 20
Value of y: 42
```


QUESTIONS

Q1

Producer-Consumer with
BlockingQueue
25 marks

Q2

Finally Block Overriding Return
Values
25 marks

Q3

try-with-resources Misuse
25 marks

Q4

wait() and notify() Coordination
25 marks

4/4 answered

Q3 try-with-resources Misuse

25 marks

```
import java.io.*;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new FileReader("data.txt"));
        try (reader) {
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            e.printStackTrace();
        }

        try (FileWriter writer = new FileWriter("out.txt")) {
            writer.write("Hello");
        }

        class NotCloseable {
            void write() {}
        }

        try (NotCloseable nc = new NotCloseable()) {
            nc.write();
        }

        try (FileReader fr1 = new FileReader("a.txt");
            FileReader fr2 = new FileReader("b.txt"))
        {
            fr1.close();
            fr2.close(); close (double-close, though usually harmless it's a logic bug/anti-pattern here compounded by bug 5)
        } catch (IOException e) {
        } finally {
            fr1.close();
        }
    }
}
```

Your Code

```
1 import java.io.*;
2 public class TryWithResourcesDemo {
3     public static void main(String[] args) {
4         try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
5             String line = reader.readLine();
6             System.out.println(line);
7         } catch (IOException e) {
8             System.out.println("Could not read data.txt: " + e.getMessage());
9         }
10        try (FileWriter writer = new FileWriter("out.txt")) {
11            writer.write("Hello");
12            System.out.println("Successfully wrote to out.txt");
13        } catch (IOException e) {
14            System.out.println("Could not write to out.txt: " + e.getMessage());
15        }
16        class NotCloseable implements AutoCloseable {
17            void write() {}
18        }
19    }
20 }
```

Input

stdin input

Output

```
Could not read data.txt:
data.txt (The system cannot
find the file specified)
Successfully wrote to out.txt
Writing...
NotCloseable closed
```


CO3-AT1-Debugging

[← Back](#)

QUESTIONS

Q1
Producer-Consumer with
Blocking Queue
25 marks**Q2**
Finally Block Overriding Return
Value
25 marks**Q3**
try-with-resources Misuse
25 marks**Q4**
wait() and notify() Coordination
25 marks

4/4 answered

Q4 wait() and notify() Coordination

25 marks

```
public class WaitNotifyDemo {  
    private boolean dataReady = false;  
    private final Object lock = new Object();  
  
    public void produce() {  
        synchronized (lock) {  
            dataReady = true;  
            System.out.println("Data produced");  
            lock.notify();  
        }  
    }  
  
    public void consume() {  
        if (!dataReady) {  
            try {  
                lock.wait();  
            } catch (InterruptedException e) {}  
        }  
        System.out.println("Data consumed");  
    }  
  
    public void consumeCorrectly() {  
        synchronized (lock) {  
            if (!dataReady) {  
                try {  
                    lock.wait();  
                } catch (InterruptedException e) {}  
                Thread.interrupted();  
            }  
        }  
        System.out.println("Data consumed");  
    }  
  
    public static void main(String[] args) {  
        WaitNotifyDemo demo = new WaitNotifyDemo();  
        Thread producer = new Thread(demo::produce);  
        Thread consumer = new Thread(demo::consumeCorrectly);  
        consumer.start();  
        producer.start();  
        consumer.join();  
    }  
}
```

Your Code

```
1 public class waitnotifydemo {  
2     private boolean dataready = false;  
3     private final Object lock = new Object();  
4     public void produce() {  
5         synchronized (lock) {  
6             dataready = true;  
7             System.out.println("data produced");  
8             lock.notify();  
9         }  
10    }  
11    public void consume() {  
12        synchronized (lock) {  
13            if (!dataready) {  
14                try {  
15                    lock.wait();  
16                } catch (InterruptedException e) {  
17                    Thread.currentThread().interrupt();  
18                }  
19            }  
20            System.out.println("data consumed");  
21        }  
22    }  
23    public void consumeCorrectly() {  
24        synchronized (lock) {  
25            if (!dataready) {  
26                try {  
27                    lock.wait();  
28                } catch (InterruptedException e) {  
29                    Thread.currentThread().interrupt();  
30                }  
31            }  
32            System.out.println("data consumed");  
33        }  
34    }  
35    public static void main(String[] args) throws InterruptedException {  
36        waitnotifydemo demo = new waitnotifydemo();  
37        Thread producer = new Thread(demo::produce);
```

Input

stdin input

Output

data produced
data consumed